

9

Extending Video Playback in Your Packtfix App

Have you ever wanted your users to continue enjoying their favorite videos even when they switch apps or turn off the screen? This chapter dives deep into the world of extended video playback on Android, bringing you the skills to create a more engaging and versatile user experience.

We'll be exploring two key functionalities: **picture-in-picture (PiP)** mode and media casting. With PiP, you'll learn how to create a miniature video player that overlays other apps, allowing users to keep an eye on the video while multitasking. With media casting, we'll use **MediaRouter** and the Cast SDK, which enable users to transfer the video playback to a larger screen, such as a TV with Google Chromecast.

By the end of this chapter, you'll have gained a solid understanding of the PiP functionalities and unlocked the potential of extended video playback in our Android app.

So, in this chapter, we will cover the following topics:

- Getting to know the PiP API
- Using PiP to continue playback in the background
- Getting to know **MediaRouter**
- Connecting to Google Chromecast devices

Technical requirements

As in the previous chapter, you will need to have installed Android Studio (or another editor of your preference).

We will follow the project started in [Chapter 7](#) with the changes we have made in [Chapter 8](#).

You can find the complete code that we are going to build through this chapter available in this repository:

<https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-9>

Getting to know the PiP API

The first step on our extended video playback journey is to understand the PiP API, which lets us use PiP mode. **PiP mode** allows users to minimize your app and continue watching a video in a resizable and movable miniature player. This functionality enhances user experience by providing flexibility and convenience.

This section will provide you with the knowledge to leverage PiP effectively in your app. We'll cover the most important aspects such as understanding the PiP requirements and learning how to enter and exit PiP mode programmatically, and review some different listener events. So, let's get started.

PiP requirements

Not every device is created equal when it comes to PiP. Before we go deep into the exciting functionalities, let's ensure a smooth user experience by understanding the requirements and compatibility aspects of PiP mode.

Regarding the requirements, there are two variables to take into account:

- **Minimum Android version:** PiP mode relies on specific APIs introduced in Android 8.0 (Oreo). Targeting devices running older versions of Android will not only prevent PiP functionality but could also lead to crashes or unexpected behavior.

To check if the user's device is compatible with PiP, we could implement the following code:

```
val minApiLevel = Build.VERSION_CODES.O
if (android.os.Build.VERSION.SDK_INT < minApiLevel) {
    // PiP not supported on this device
    return false
}
```

This code ensures our app gracefully handles devices that can't use PiP mode. First, we define the minimum Android version required for PiP (typically Android 8.0 or Oreo). Then, we check the device's current version. If it's older than the minimum, the code recognizes that PiP functionality isn't available and signals this back (potentially by returning **false**) to prevent the app from attempting to use PiP features that would cause issues on incompatible devices.

This allows you to gracefully handle situations where PiP isn't available and potentially offer alternative functionalities for users on older devices (for example, we could offer them to send the playback to a different device).

- **Screen size requirements:** While PiP mode can be technically implemented on various screen sizes, smaller displays might not provide an optimal user experience. Imagine trying to watch a movie in a tiny PiP window on a phone with a 4-inch screen! Therefore, it's essential to consider screen size limitations.

Now that we've established the requirements, let's explore the exciting part: initiating PiP mode within our app.

Entering and exiting PiP mode programmatically

As we already know, PiP mode offers users the convenience of continuing video playback in a miniature window even when they switch apps or turn off the screen. To do this, we'll use the `enterPictureInPictureMode()` method available in the `Activity` class:

```
activity.enterPictureInPictureMode()
```

Calling this method allows you to programmatically trigger PiP mode from within your activity, and the system will handle resizing the video player window and placing it on top of other apps. It's important to note that you should typically only call this method when the user explicitly requests it, such as upon tapping a dedicated PiP button within your app's UI.

While entering PiP mode is initiated programmatically, exiting is primarily user-driven. The user can exit PiP mode by swiping the miniature player away or tapping a designated **Close** button provided by the system. However, as developers, we can still play a role in ensuring a smooth transition back to the fullscreen experience. The system triggers specific callbacks within your activity when PiP mode is exited. Here's how we can utilize these callbacks:

```
override fun onPictureInPictureExited() {  
    super.onPictureInPictureExited()  
    // Any logic that we want to add when the user comes back  
    // to the full screen experience in our app  
}
```

This function will be called every time the user closes the PiP miniature screen. This is not the only function that we can use to handle PiP status changes, though; the listener provides various events to keep our app informed about changes in the PiP window. These events allow us to react and update our app's behavior accordingly, ensuring a seamless user experience:

- **onPictureInPictureEntered()**: This event gets triggered when the user successfully enters PiP mode. You can use this opportunity to potentially update UI elements to reflect the PiP state (for example, hide unnecessary controls) or perform any necessary optimizations for PiP playback (for example, adjust video quality).
- **onPictureInPictureExited()**: As discussed previously, this event signifies the user exiting PiP mode. Here, you can clean up resources associated with the PiP window or update the UI to reflect the return to fullscreen playback.
- **onPictureInPictureUiStateChanged()**: This event gets fired whenever any change occurs to the PiP window, such as resizing or moving it. You might use this to adjust your UI layout based on the new PiP

window dimensions or update video playback based on potential performance changes due to resizing.

By effectively handling PiP events and listener callbacks, you can keep your app in sync with the changing PiP window state. Now, let's see how we can integrate it into our existing project.

Using PiP to continue playback in the background

The first step before we can use PiP in our project is that we must declare support for it in our **AndroidManifest.xml** file. This step is crucial for informing the Android system that our **PlaybackActivity** class is capable of running in PiP mode. We do this like so:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
    xmlns:android =
        "http://schemas.android.com/apk/res/android">
    <application>
        <activity
            android:name = "com.packt.playback.presentation
                .PlaybackActivity"
            android:supportsPictureInPicture="true"
            android:resizeableActivity="true"
            android:screenOrientation="landscape"/>
        </application>
    </manifest>
```

For PiP specifically, the key attribute in our manifest is **android:supportsPictureInPicture="true"**, which explicitly declares that your activity supports PiP mode.

The **resizeableActivity** attribute, while related to the ability of an activity to be resized, is implicitly set to **true** for all activities when targeting API level 24 or higher. This means if your app targets API level 24+, you don't need to explicitly set **resizeableActivity="true"** for PiP mode to work because the system already considers all activities to be resizable to support multi-window mode.

However, explicitly setting **resizeableActivity="true"** can be a good practice for clarity, especially if your app is designed to take advantage of multi-window features beyond just PiP, or if you want to ensure compatibility across different Android versions and devices. It's also useful for documentation purposes, making it clear to anyone reading your **AndroidManifest.xml** file that your activity is intended to support resizable behaviors, including PiP.

Implementing PiP

Now that we have explicitly opted in our **Activity** class to use the PiP feature, let's implement it. We will override the **onUserLeaveHint()** callback, which is triggered when the user presses the **Home** button or switches to another app:

```
override fun onUserLeaveHint() {  
    super.onUserLeaveHint()  
    val aspectRatio = Rational(16, 9)  
    val params = PictureInPictureParams.Builder()  
        .setAspectRatio(aspectRatio)  
        .build()  
    enterPictureInPictureMode(params)  
}
```

As we said, we are overriding the **onUserLeaveHint()** existing function. Here, we still have to include the call to **super.onUserLeaveHint()** as it ensures that the **Activity** class properly handles any additional underlying operations defined by Android's framework before executing custom behavior.

Within this method, the aspect ratio for the PiP window is defined as **16:9**, a common choice for video content, by using the **Rational** class. This aspect ratio is crucial as it dictates the proportional relationship between the width and height of the PiP window, ensuring the video maintains its intended appearance without distortion.

To apply this aspect ratio, the **PictureInPictureParams.Builder** class is utilized to construct a configuration object. By invoking **setAspectRatio(aspectRatio)** on the builder, the previously defined aspect ratio is applied to this configuration.

While **setAspectRatio(Rational)** sets the preferred aspect ratio of the PiP window, meaning that the system will try to maintain this aspect ratio when displaying the PiP window, it may not always be possible depending on the device and screen size constraints. Android 11 (API level 30) introduced **setMaxAspectRatio(Rational)** and **setMinAspectRatio(Rational)** for defining the maximum and minimum aspect ratios. Additionally, **setMaxSize(int, int)** allows setting the maximum size of the PiP window, providing greater control over how the PiP window appears on different devices.

NOTE

*There are also other **PictureInPictureParams.Builder** options that could be applied. For more information about these options, refer to the documentation:*

<https://developer.android.com/reference/android/app/PictureInPictureParams.Builder>.

The **build()** method then compiles these configurations into a **PictureInPictureParams** object, which encapsulates all the necessary settings for entering PiP mode.

Finally, the `enterPictureInPictureMode(params)` method is invoked, signaling the system to transition the current **Activity** class into PiP mode using the specified parameters.

Now that we have integrated this feature, when we are on the playback screen and we leave the application, we should still see the video on the PiP screen:

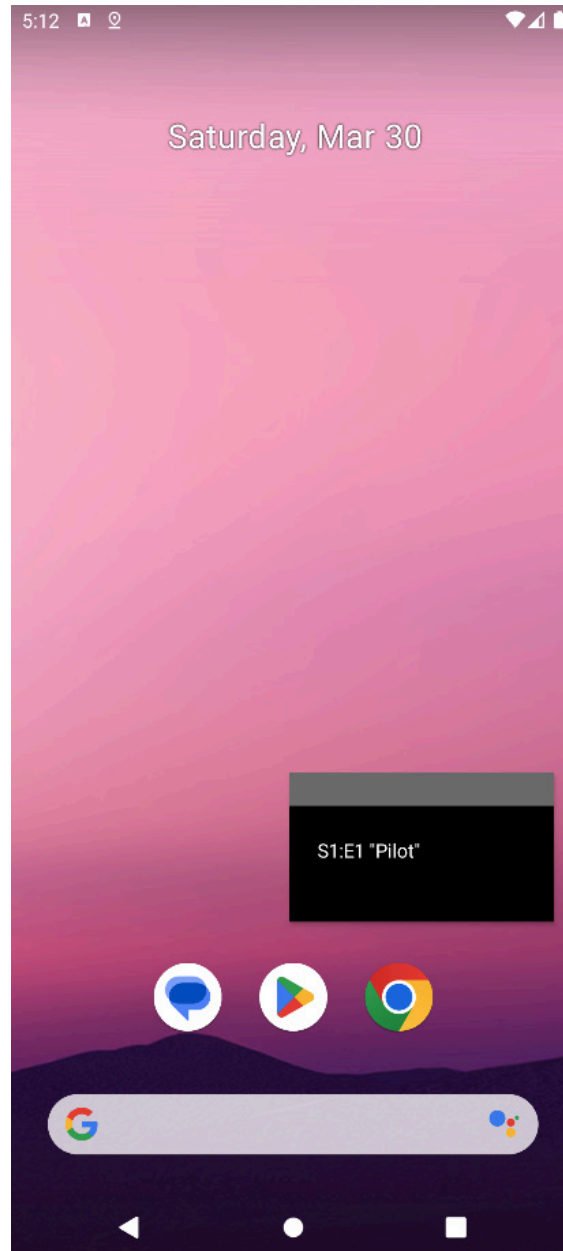


Figure 9.1: Playback using the PiP feature

The `PictureInPictureParams.Builder` class in Android provides a customizable way to configure the behavior and appearance of an app when it enters PiP mode. Apart from setting the aspect ratio with `setAspectRatio()`, as we did in the previous instruction, there are several other options available to tailor the PiP experience:

- **Actions:** Using `setActions(List<RemoteAction>)`, developers can specify a list of actions that the user can perform while in PiP mode.

These actions are represented as **RemoteAction** objects and can include things such as play, pause, or skip. These actions appear as buttons in the PiP window, providing interactive elements for the user without needing to return to the full app interface.

- **Auto enter/exit:** Through **setAutoEnterEnabled(boolean)** and **setAutoExitEnabled(boolean)** (introduced in later Android versions), developers can control whether the app should automatically enter or exit PiP mode based on certain conditions, such as media playback state.
- **Seamless resize:** By invoking **setSeamlessResizeEnabled(boolean)**, it's possible to enable or disable seamless resizing for the PiP window. This option, available in later Android versions, helps make the transition into and out of PiP mode smoother visually.
- **Source rect hint:** **setSourceRectHint(Rect)** allows developers to suggest a preferred area of the screen that the PiP mode should try to align with when entering PiP mode. This can be useful for guiding the system on where the PiP window should ideally be placed based on the app's UI layout.

Let's use these options to add actions so that the user can toggle between play and pause in the PiP view. But first, a little theory.

Understanding how to add actions to the PiP mode

Integrating actions into PiP mode enhances user interaction by allowing direct control over app functionality without leaving the PiP window. By using the **setActions(List<RemoteAction>)** method, you can create a more immersive and user-friendly experience, offering controls such as play, pause, or skip directly within the PiP overlay. This capability is especially valuable in media applications, where users often need to manage playback without disrupting their current onscreen activities.

In a moment, we will learn how to effectively create and manage these **RemoteAction** objects, ensuring our app's PiP mode is both functional and engaging, complementing the existing array of PiP features. But let's dig into the concepts further.

Each **RemoteAction** object represents an actionable element in the PiP window, such as a button for play, pause, or skip functionality. To create these actions, we would have to specify an icon, a title, a **PendingIntent** object that defines the action to take when the user interacts with the button, and a description for accessibility purposes.

The utilization of a **PendingIntent** object is crucial here, as it allows the action to trigger specific behaviors in your app when invoked. An **Intent** object in Android is like a message that can signify a wide range of events, including system boot completion, network changes, or custom events defined by the application. Typically, these intents are directed toward a **BroadcastReceiver** instance within your application.

A **BroadcastReceiver** instance in Android is a fundamental component that enables applications to listen for and respond to broadcast messages from other applications or from the system itself. When an intent that matches a **BroadcastReceiver** instance's filter is broadcasted, the **BroadcastReceiver** instance's **onReceive()** method is invoked, allowing the app to execute logic in response to the event. This mechanism provides a powerful way for applications to react to global system events or inter-app communication without needing to be running in the foreground, making **BroadcastReceiver** instances a key tool for event-driven programming in Android.

In our case, this **BroadcastReceiver** instance is responsible for listening to and processing the broadcasted intents sent by PiP actions. For instance, when a user presses the **Play** button in the PiP window, the **PendingIntent** object associated with the play action is broadcasted, and the corresponding receiver in your app catches this intent and triggers the media to play.

The need for a **BroadcastReceiver** instance arises from the decoupled nature of PiP action intents from direct method calls within your app. Since these actions occur outside the regular UI flow, using a broadcast mechanism allows your app to respond to these actions asynchronously and perform the necessary operations, such as updating the media playback state. This setup ensures that your app can handle PiP controls effectively, providing a seamless experience for users even when interacting with the app from the PiP window.

Now that we know how to create **RemoteAction** objects, let's apply our learnings in our project.

Adding actions to the PiP mode

Let's start by creating our **BroadcastReceiver** subclass. This class will extend **BroadcastReceiver** and override the **onReceive()** method, where you'll define how your app should react to PiP action **Intent** objects:

```
class PiPActionReceiver(private val togglePlayPause: () -> Unit) : BroadcastReceiver() {
    override fun onReceive(context: Context?, intent:
        Intent?) {
        when (intent?.action) {
            ACTION_TOGGLE_PLAY -> {
                togglePlayPause()
            }
        }
    }
    companion object {
        const val ACTION_TOGGLE_PLAY =
            "com.packflix.action.TOGGLE_PLAY"
    }
}
```


In the **onReceive** method, a check is performed on the **Intent** action to determine if it matches the **ACTION_TOGGLE_PLAY** action. If it does, the play/pause toggle logic will be executed. In this case, we will execute a callback, as the logic to play or pause the playback will likely be outside this receiver.

Next, we need to register the **BroadcastReceiver** instance so that it can receive the **Intent** object. This can be done in two ways:

- **Manifest declaration:** Registering in the **AndroidManifest.xml** file is suitable for actions that should be received even if your app is not running. However, for PiP actions, dynamic registration in the activity or service that handles PiP mode is often more appropriate.
- **Dynamic registration:** Since PiP actions are specifically related to when our app is in PiP mode, registering the **BroadcastReceiver** instance dynamically in our **PlaybackActivity** class allows for more control and is contextually relevant.

We will register the **BroadcastReceiver** instance using dynamic registration. In our **PlaybackActivity** class, the implementation will look like this:

```
private lateinit var pipActionReceiver: PiPActionReceiver
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    pipActionReceiver = PiPActionReceiver {
        //TODO handle there the play/pause logic
    }
    val filter =
        IntentFilter(PiPActionReceiver.ACTION_TOGGLE_PLAY)
    if (Build.VERSION.SDK_INT >=
        Build.VERSION_CODES.TIRAMISU) {
        registerReceiver(pipActionReceiver, filter,
            RECEIVER_NOT_EXPORTED)
    } else {
        registerReceiver(pipActionReceiver, filter)
    }
    setContent {
        PlaybackScreen()
    }
}
```

First, we will declare a **BroadcastReceiver** variable named **pipActionReceiver**. This receiver is not initialized immediately (it is declared as **lateinit var**) because it will be set up in the **onCreate** method of our activity.

In the **onCreate** method, we will initialize the **BroadcastReceiver** variable. The **pipActionReceiver** variable is instantiated and assigned a lambda function as its argument. This function is intended to contain the logic that handles the play/pause action.

Then, we will register the **BroadcastReceiver** variable, indicating the **Intent** filter signal it will listen to. The registration method differs depending on the SDK version:

- For SDK versions Tiramisu (Android 13, API level 33) and above, you use the **registerReceiver** method with an additional flag, **RECEIVER_NOT_EXPORTED**, for enhanced security, ensuring that your receiver does not inadvertently become accessible to other apps.
- For earlier versions, you register the receiver without this flag. This ensures backward compatibility while adhering to best practices for app security on newer devices.

Now, let's create the action that will trigger the **Intent** action needed to launch the **BroadcastReceiver** instance:

```
private fun getIntentForTogglePlayPauseAction():
    RemoteAction {
        val icon: Icon = Icon.createWithResource(this,
            R.drawable.baseline_play_arrow_24)
        val intent =
            Intent(PiActionReceiver.ACTION_TOGGLE_PLAY).let {
                intent ->
                    PendingIntent.getBroadcast(this, 0, intent,
                        PendingIntent.FLAG_UPDATE_CURRENT or
                        PendingIntent.FLAG_IMMUTABLE)
            }
        return RemoteAction(icon, "Toggle Play", "Play or pause
            the video", intent)
    }
```

In this code, we are creating a **RemoteAction** method. The first line inside the method creates an **Icon** object from a drawable resource (**R.drawable.baseline_play_arrow_24**). This icon visually represents the toggle play/pause action to the user.

Then, a new **Intent** object is instantiated with the **PiActionReceiver.ACTION_TOGGLE_PLAY** action. This **Intent** object is designed to be broadcasted when the **RemoteAction** method is invoked by the user. The **let** block is utilized to directly chain the creation of a **PendingIntent** object that wraps this **Intent** object, making it executable from outside the application context.

The **PendingIntent.getBroadcast** method is called to create a **PendingIntent** object that broadcasts the **Intent** object. This **PendingIntent** object is configured with **PendingIntent.FLAG_UPDATE_CURRENT** to ensure that if the pending **Intent** object already exists, it will be reused but with its extra data updated. **PendingIntent.FLAG_IMMUTABLE** is used for security purposes, marking the **Intent** object as immutable to prevent alterations after creation.

Finally, a **RemoteAction** object is instantiated and returned. This object takes the previously created icon, a title (**Toggle Play**), a content description (**Play or pause the video**), and the **PendingIntent** object as its parameters. The title and content description should be concise yet descriptive enough to inform the user of the action's purpose, adhering to accessibility standards.

Now, we need to configure this action as a parameter for our PiP configuration. We will modify the existing configuration as follows:

```
override fun onUserLeaveHint() {
    super.onUserLeaveHint()
    val aspectRatio = Rational(16, 9)
    val params = PictureInPictureParams.Builder()
        .setAspectRatio(aspectRatio)
        .setActions(listOf(
            getIntentForTogglePlayPauseAction()))
        .build()
    enterPictureInPictureMode(params)
}
```

Here, we are using the **setActions()** function to add a list including the new action.

The last step is to handle the logic to effectively toggle between play and pause. We already have this functionality implemented in the **ViewModel** component, so we just have to inject the **PlaybackViewModel** component in the **Activity** class and call the **togglePlayPause()** function:

```
@AndroidEntryPoint
class PlaybackActivity: ComponentActivity() {
    private val viewModel: PlaybackViewModel by
        viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        pipActionReceiver = PiPActionReceiver {
            viewModel.togglePlayPause()
        }
        ...
    }
}
```

As we can see, we are injecting **PlaybackViewModel** and then, the **viewModel.toggle PlayPause()** function will be invoked when the receiver detects that the user has sent a broadcast with the play/pause action.

If we execute the code with these changes, we should see the **Play** button in our PiP UI:

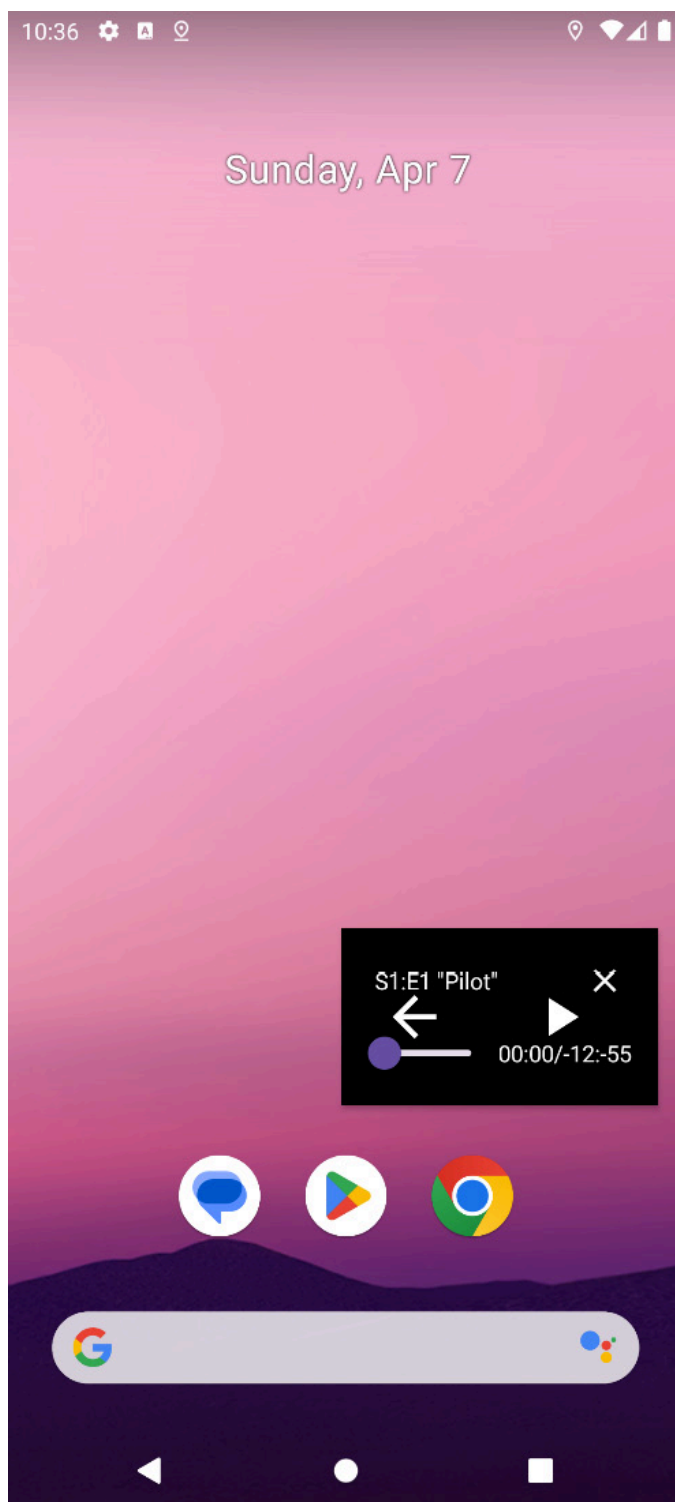


Figure 9.2: PiP view with some actions

Having implemented PiP mode, let's move on to connecting with other devices for media playback using the **MediaRouter** API, which allows your app to cast or stream content to devices such as smart TVs or Chromecast. We'll cover how to use **MediaRouter** to identify compatible devices and manage media streaming to them, enhancing our app's functionality.

Getting to know MediaRouter

MediaRouter is a pivotal component in Android development, especially for applications that deal with multimedia content. It acts as a bridge between devices running your app and external devices such as Google Chromecast, smart TVs, and various speakers that support media routing capabilities.

The core function of **MediaRouter** is to facilitate the streaming of multimedia content—be it audio, video, or images—from the user's current device to another device that provides a better or more suitable playback experience. It intelligently discovers available media routes and allows the application to connect to them, thereby extending the multimedia capabilities beyond the confines of the user's primary device.

Android's **MediaRouter** API provides a framework that developers can utilize to search for and interact with media route providers registered on the local network. These providers represent devices or services capable of media playback. With **MediaRouter**, applications can not only discover these routes dynamically but also present the user with a streamlined interface for choosing their preferred playback devices, all while managing the connections and playback state seamlessly across devices.

The use of **MediaRouter** in Android apps opens up a myriad of possibilities for enhancing the user's media consumption experience. Here are some typical use cases:

- **Casting videos to larger screens:** One of the most common uses of **MediaRouter** is casting videos from a mobile device to a larger display, such as a smart TV or a monitor with Chromecast. This is particularly appealing for watching movies, TV shows, or user-generated content on a bigger screen that offers a more immersive viewing experience.
- **Streaming music to external speakers:** **MediaRouter** allows apps to stream music to external speakers, amplifying the audio experience. This is ideal for parties, workouts, or simply enhancing the quality of music playback beyond what the phone's or tablet's built-in speakers can provide.
- **Displaying images on a shared screen:** Apps can use **MediaRouter** to send images to a smart TV or a connected display, making it perfect for sharing photos with a group, conducting presentations, or viewing artwork in higher resolution.
- **Gaming:** With the capability to cast screen content to a larger display, gaming apps can leverage **MediaRouter** to provide a console-like gaming experience on the TV while using the mobile device as a controller.
- **Fitness and education:** For apps focused on fitness or education, casting instructional videos or workout routines to a TV allows users to follow along more comfortably and effectively.

In each of these use cases, **MediaRouter** significantly enhances the functionality of apps by leveraging the power of connected devices, thus offering users a more flexible and enriched media playback experience. Through its comprehensive API, developers can create applications that are not just confined to the small screens of mobile devices but are in-

stead capable of bringing content to life on any compatible device within the home network.

Setting up MediaRouter

Integrating **MediaRouter** into our Android app involves a few key setup steps, including adding the necessary dependencies to your project and ensuring you have the correct permissions in place.

First, we'll need to include the **MediaRouter** library dependencies in our **libs.versions.toml** file. This library provides the classes and interfaces needed to discover and interact with media route providers:

```
[versions]
...
mediarouter = "1.7.0"
google-cast = "21.4.0"
[libraries]
...
media-router = { group = "androidx.mediarouter", name="mediarouter", version.ref="mediarouter"}
google-cast = { group = "com.google.android.gms", name="play-services-cast-framework", version.ref="google-cast" }
```

As we plan to support casting to Chromecast devices or other Google Cast-enabled devices, we need the **play-services-cast-framework** library. This library facilitates the integration with Google Cast devices and extends the capabilities of **MediaRouter**.

The next step will be to add it to our **build.gradle** module:

```
implementation(libs.media.router)
implementation(libs.google.cast)
```

Now, to enable **MediaRouter** to discover and interact with devices on the local network, we must declare the necessary permissions in our app's **AndroidManifest.xml** file:

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

We are including permissions here for the following:

- **Internet permission:** Since **MediaRouter** may use the network to communicate with media route providers, your app needs permission to access the internet. We have already needed to declare this permission for the previous chapters, so it shouldn't be new.
- **Network state permissions:** These permissions are required for the app to monitor changes in network connectivity, which is essential for discovering devices on the network.

- **Local network permissions (Android 12 and above):** Starting with Android 12 (API level 31), if your app targets API level 31 or higher and needs to discover devices on the local network, you must also declare the permission.
- **Post notifications:** For Android 12+, to access the local network for device discovery, it is mandatory to have this permission.

After adding the necessary dependencies and permissions, our project is ready to use **MediaRouter** for discovering media route providers and enabling media streaming to external devices.

Discovering media routes

Once your app is set up with the necessary **MediaRouter** dependencies and permissions, the next step is discovering available media routes. This involves identifying external devices or services your app can stream media to. Android's **MediaRouter** framework simplifies this by providing tools to both discover media routes and present them to users.

Learning about **MediaRouteProvider**

MediaRouteProvider is a component that publishes media routes to **MediaRouter**. It acts as a bridge between your app and external devices or services, such as speakers, TVs, or other Cast-enabled devices. There are two options to use **MediaRouteProvider**:

- The default **MediaRouteProvider** implementation: For most use cases, especially when integrating with Google Cast devices, Android provides a default **MediaRouteProvider** implementation, so you don't need to implement your own. The Google Cast framework automatically discovers compatible devices and makes them available as media routes.
- A custom **MediaRouteProvider** implementation: If you need to discover devices for a custom protocol or a specific type of media routing not covered by Google Cast, you can implement your own **MediaRouteProvider** instance by extending the **MediaRouteProvider** class. This involves defining the discovery logic and publishing routes to **MediaRouter**.

However, creating a custom **MediaRouteProvider** implementation is beyond the scope of basic media routing and requires in-depth knowledge of the specific hardware or protocol you're targeting. If you want to know more, here is the official documentation to create a customized **MediaRouteProvider** implementation:

<https://developer.android.com/media/routing/mediarouteprovider>.

We will use the default **MediaRouteProvider** implementation instead.

Using the **MediaRouter** class

The **MediaRouter** class is your primary tool for interacting with media routes. Here's how you can use it to discover and monitor available me-

dia routes.

We will begin by defining a **MediaRouteSelector** instance and allow it to start discovering other devices to send the media to. We will use **LaunchedEffect** to tie the discovery process to the composable's lifecycle:

```
@Composable
fun MediaRouteDiscoveryOptions(mediaRouter: MediaRouter) {
    val context = LocalContext.current
    val routeSelector = remember {
        MediaRouteSelector.Builder()
            .addControlCategory(
                MediaControlIntent.CATEGORY_REMOTE_PLAYBACK
            )
            .build()
    }
    val mediaRoutes = remember {
        mutableStateListOf<MediaRouter.RouteInfo>()
    }
    DisposableEffect(mediaRouter) {
        mediaRouter.addCallback(routeSelector, callback,
            MediaRouter.CALLBACK_FLAG_PERFORM_ACTIVE_SCAN)
        onDispose {
            mediaRouter.removeCallback(callback)
        }
    }
}
```

This composable function accepts a **MediaRouter** instance as a parameter, highlighting its dependency on this framework for discovering media routes.

The function begins by obtaining the current **Context** value using **LocalContext.current**, then it creates a **MediaRouterSelector** instance. This selector is specifically configured to filter for routes supporting live video content. The use of **remember** ensures that the **MediaRouteSelector** instance is preserved across recompositions of the composable, optimizing performance by preventing unnecessary reinitializations.

Then, we are adding a **DisposableEffect** composable, which encapsulates the logic for starting and stopping media route discovery in alignment with the composable's lifecycle. By passing **MediaRouter** as a key to **DisposableEffect**, the enclosed block of code is executed in a coroutine when the composable is first composed into the UI, and the coroutine is canceled when the composable is removed, effectively managing the lifecycle of the discovery process. Within this block, the **addCallback** method of **MediaRouter** is called to register a callback with the active scan flag, initiating the active scanning for media routes that match the criteria set by **routeSelector**. The **onDispose** block within **DisposableEffect** serves as a cleanup mechanism, where the callback is unregistered from **MediaRouter** when the composable is disposed of, ensuring resources are freed and background processing is minimized.

Now, we will create a callback that we have included in the **addCallback** function described previously:

```
val callback = remember {
    object : MediaRouter.Callback() {
        override fun onRouteAdded(router: MediaRouter,
            route: MediaRouter.RouteInfo) {
            mediaRoutes.add(route)
        }
        override fun onRouteRemoved(router: MediaRouter,
            route: MediaRouter.RouteInfo) {
            mediaRoutes.remove(route)
        }
    }
}
```

We are instantiating a **MediaRouter.Callback** listener, using **remember** to avoid needing to recreate it every time the app's UI updates.

This listener, **MediaRouter.Callback**, has two main jobs through its **onRouteAdded** and **onRouteRemoved** methods. When a new device becomes available for casting media, **onRouteAdded** gets called, and the app adds this new route to a list called **mediaRoutes**. This list is crucial for the app to know what devices are available at any moment. On the flip side, when a device goes offline or disconnects, **onRouteRemoved** is called, and the app removes that route from the list, ensuring the list stays current.

Effectively, this setup allows the app to dynamically adjust to changes in the available devices for media casting.

To provide users with an easy way to select from these available devices, we need to integrate a button designed for this purpose. The **MediaRouter** API offers a ready-made button that displays the available devices for casting. Although this button is an Android view and not a composable, we can still use it in Jetpack Compose by wrapping it with the **AndroidView** composable. Here's how we can do it:

```
AndroidView(
    factory = { ctx ->
        MediaRouteButton(ctx).apply {
            setRouteSelector(routeSelector)
        }
    },
    modifier = Modifier
        .wrapContentWidth()
        .wrapContentHeight()
)
```

Now, we just have to use the **MediaRouteDiscoveryOptions** composable from our playback screen:

```
@Composable
```

```
fun TopMediaRow(mediaRouter: MediaRouter, modifier:
Modifier = Modifier) {
    Row(
        modifier = modifier
        .fillMaxWidth()
        .padding(20.dp),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = "S1:E1 \"Pilot\"", color = Color.White)
        MediaRouteDiscoveryOptions(mediaRouter =
            mediaRouter)
    }
}
```

Here, we have added the **MediaRouteDiscoveryOptions** composable to our already existing **TopMediaRow** function.

When calling the **TopMediaRow** function we will pass it an instance of **mediaRouter** that we had obtained before, using **LocalContext**:

```
TopMediaRow(
    mediaRouter =
        MediaRouter.getInstance(LocalContext.current),
    modifier = Modifier.align(Alignment.TopCenter))
```

Now, we will see the **Cast** button in the right corner of our existing **PlaybackScreen** composable. If we click it, **MediaRouter** will automatically search for devices:

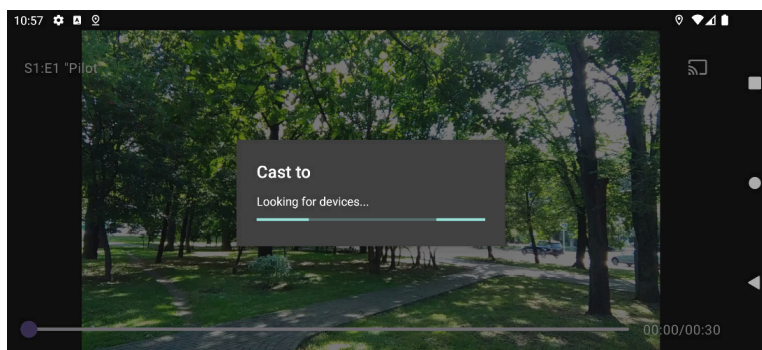


Figure 9.3: MediaRouter searching for devices

If it cannot find any device, it will show a message encouraging the user to check the connection:

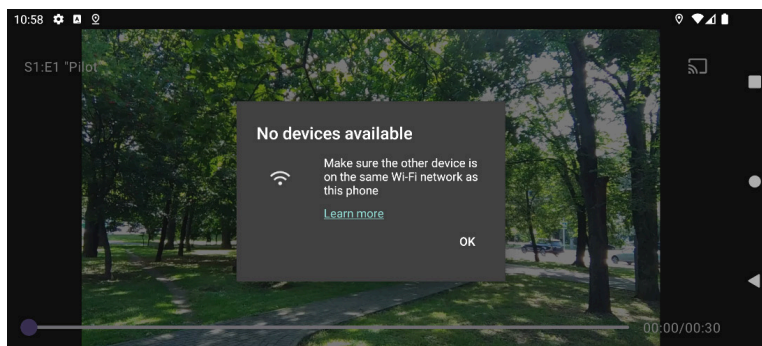


Figure 9.4: MediaRouter functionality, prompting the user to check the device connections

After discovering available media routes using the **MediaRouter** API, the next step is connecting to a selected device for media playback. This involves two main actions: selecting a media route and then establishing a connection to that route. Here's how you can approach this process.

When utilizing the built-in media route selector with **MediaRouteButton**, the process of connecting to a device is streamlined. **MediaRouteButton** automatically handles the display of available media routes based on the criteria defined in a **MediaRouteSelector** instance. Users can then select their preferred device directly from the UI that **MediaRouteButton** presents.

Once a user selects a route from the dialog, the connection to that device is automatically managed by the **MediaRouter** framework based on the route's capabilities and the types of media specified in your **MediaRouteSelector** instance. There's no need for additional manual connection management in your application code.

With the route selected and a connection established, you can control media playback through the selected route. This typically involves using media control APIs that are appropriate for your application's media content and the capabilities of the selected route. We will learn how we can cast media playback for Google Cast devices in the next section.

Connecting to Google Chromecast devices

Google Cast is a powerful technology developed by Google that allows users to wirelessly stream audio and video content from their smartphones, tablets, or computers directly to Cast-enabled devices. This technology is embedded in a wide array of devices, including Chromecast dongles, smart TVs, and speakers, making it accessible to a vast user base. At its core, Google Cast works by establishing a connection between a Cast-enabled app on a mobile device or computer and a Cast-enabled receiver device. Once a connection is made, media can be played back on the receiver device, effectively turning it into a remote screen or speaker for the content being cast.

The functionality of Google Cast is not limited to streaming media from the internet. It also enables the mirroring of content from the sender device's screen, extending its utility to presentations, educational content, and more. Google Cast operates over Wi-Fi, ensuring high-quality streaming performance without the need for physical cables or adapters.

We have already done some steps: we have already included the library and we are already detecting the devices that allow casting. Now, we need to establish a cast session. This session facilitates a connection between

your app and the selected Cast device, enabling media control and playback on the larger screen. This process hinges on effectively using **CastContext** and adeptly managing Cast session events.

CastContext is central to initiating and managing Cast sessions in your application, providing the necessary APIs to connect to the selected Cast device. Here's how to initiate a connection.

First, we need to ensure that you have initialized **CastContext** in your application. This is typically done in the **Application** subclass or your main activity. We will initialize it in our **PlaybackActivity** class:

```
val castContext = CastContext.getSharedInstance(context)
```

Then, we need to select a device. We have already implemented **MediaRouterbutton**, which will automatically handle the selection. Once a device is selected, the Cast SDK automatically initiates a connection to the device. This process is abstracted away from the developer, but it's crucial to listen for session events to manage the connection effectively.

The Cast SDK provides callbacks for session events such as starting, ending, resuming, and suspending. Handling these events allows your app to respond to changes in the session state, such as updating the UI or pausing media playback when the session ends.

To listen to these session events, we must implement **SessionManagerListener**:

```
private val sessionManagerListener = object : SessionManagerListener<CastSession> {  
    override fun onSessionStarted(session: CastSession,  
        sessionId: String) {  
        castSession = session  
        updateUIForCastSession(true)  
    }  
    override fun onSessionEnded(p0: CastSession, p1: Int) {  
        castSession = null  
        updateUIForCastSession(false)  
    }  
    override fun onSessionResumed(session: CastSession, p1:  
        Boolean) {  
        castSession = session  
        updateUIForCastSession(true)  
    }  
    override fun onSessionStarting(p0: CastSession) {}  
    override fun onSessionStartFailed(  
        p0: CastSession, p1: Int) {}  
    override fun onSessionResuming(session: CastSession,  
        p1: String) {}  
    override fun onSessionResumeFailed(session:  
        CastSession, p1: Int) { }  
    override fun onSessionEnding(session: CastSession) {}  
    override fun onSessionSuspended(p0: CastSession,
```

```

        p1: Int) {}
    }

```

Here, we are implementing our **SessionManagerListener<CastSession>** interface, crucial for managing Google Cast sessions. This listener is designed to react to various events related to the lifecycle of a Cast session, including its start, end, resumption, and failure cases. Let's look deeper into this implementation:

- **onSessionStarted:** This callback is invoked when a new Cast session has successfully started. Here, the **session** parameter, which is an instance of **CastSession**, represents the newly established session. The method sets the global **castSession** variable to this instance, effectively marking the beginning of a session. Subsequently, it calls **updateUIForCastSession(true)**, a method that will be implemented to update the application's UI to reflect that casting has started.
- **onSessionEnded:** Triggered when an existing Cast session ends, this method clears the **castSession** variable by setting it to **null**, indicating that there is no longer an active Cast session. It also invokes **updateUIForCastSession(false)** to adjust the UI, signaling to the user that casting has stopped.
- **onSessionResumed:** Similar to **onSessionStarted**, this callback is called when a previously suspended Cast session is resumed. It updates **castSession** with the current session and calls **updateUIForCastSession(true)** to reflect the resumption of casting in the UI.
- **onSessionStarting** and **onSessionResuming:** Indicate that a session is in the process of starting or resuming but has not yet completed. No action is taken in these callbacks in our case.
- **onSessionStartFailed** and **onSessionResumeFailed:** Called when attempts to start or resume a session fail. Again, no action is specified in our case, but these would be appropriate places to handle errors, such as by notifying the user or attempting to restart the session.
- **onSessionEnding** and **onSessionSuspended:** These callbacks are triggered when a session is in the process of ending or being suspended. As with the start and resume events, no specific actions are taken in these cases.

Once we have implemented our listener, we need to register it using **castContext.sessionManager**:

```

override fun onStart() {
    super.onStart()
    castContext.sessionManager.addSessionManagerListener(
        sessionManagerListener, CastSession::class.java)
}
override fun onStop() {
    super.onStop()
    castContext.sessionManager.removeSessionManagerListener(
        sessionManagerListener, CastSession::class.java)
}

```

Here, we are registering the listener when the **Activity** class is started and removing it when it is stopped. That way, we ensure that the listener is only retained when the **Activity** class is in a started state.

Now, let's implement the **updateUIForCastSession** function:

```
private fun updateUIForCastSession(isCasting: Boolean) {
    viewModel.setCastingState(isCasting)
}
```

Here, we are calling a new function that we will include next in the **ViewModel** component, called **setCastingState**. We are passing a **Boolean** as the argument, indicating whether the app is casting or not.

In our **PlaybackViewModel** component, we will introduce the following changes. We will start adding a new property, **isCasting**:

```
private val _isCasting = MutableStateFlow<Boolean>(false)
val isCasting: MutableStateFlow<Boolean> = _isCasting
```

Then, we will change its value when the **setCastingState** function is called:

```
fun setCastingState(isCasting: Boolean) {
    _isCasting.value = isCasting
}
```

Then, we will use it in our **PlaybackScreen** composable:

```
@Composable
fun PlaybackScreen() {
    ...
    val isCasting = viewModel.isCasting.collectAsState()
    Box(
        ...
    ) {
        if (isCasting.value) {
            NowCastingView()
        } else {
            //VideoPlayerComposable and the rest of the UI...
        }
    }
}
```

In our already existing **PlaybackScreen** composable, we have added a new property, **isCasting**. This property is used to choose if the screen will show a **Now Casting** message or the complete playback UI.

Next, we will build a new **NowCastingView** composable:

```
@Composable
```

```
fun NowCastingView() {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
    ) {
        Column(
            modifier = Modifier.padding(16.dp)
        ) {
            Text(
                text = "Now Casting",
                style =
                    MaterialTheme.typography.headlineMedium
            )
        }
    }
}
```

This composable is just placing and showing a text with the **Now Casting** content, just to make the user aware that the media content is currently being cast to another device.

There's just one thing that we must do: load the media in the remote device. We will modify the **onSessionStarted** callback in the **SessionManagerListener** interface, including a call to a new function to load the media:

```
override fun onSessionStarted(session: CastSession,
    sessionId: String) {
    castSession = session
    updateUIForCastSession(true)
    loadMedia(session)
}
```

Finally, we will implement this function as follows:

```
private fun loadMedia(castSession: CastSession) {
    val mediaInfo = MediaInfo.Builder(viewModel.mediaUrl)
        .setStreamType(MediaInfo.STREAM_TYPE_BUFFERED)
        .setContentType("video/mp4")
        .build()
    val mediaLoadOptions = MediaLoadOptions
        .Builder()
        .setAutoplay(true)
        .setPlayPosition(0)
        .build()
    castSession.remoteMediaClient?.load(mediaInfo,
        mediaLoadOptions)
}
```

The function begins by constructing a **MediaInfo** object, which encapsulates all necessary details about the media file intended for playback. Utilizing the **MediaInfo.Builder** pattern, it starts with specifying the media's URL, sourced from **viewModel.mediaUrl**. This URL is the location of the media file, which the Cast-enabled device will stream. The builder

then sets the stream type to `MediaInfo.STREAM_TYPE_BUFFERED`, indicating that the content is pre-recorded and can be buffered before playback, which is ideal for video content that isn't being streamed live. Furthermore, the content type is set to `"video/mp4"`, defining the **MIME** type (**M**ultipurpose **I**nternet **M**ail **E**xtensions, used not only by email but also by web browsers and apps to interpret and display content correctly.) of the file as an MP4 video.

Following the creation of the `MediaInfo` object, the function proceeds to configure additional playback options through a `MediaLoadOptions` object. The options set include `setAutoplay(true)`, which commands the Cast device to automatically start playing the media as soon as it's loaded, and `setPlayPosition(0)`, ensuring that playback commences from the very beginning of the media file, for simplicity. One improvement to this could be to obtain the current play position from the `ViewModel` component so that the video can continue at the same point in time if the playback has already started.

The final step in the `loadMedia` function involves invoking the `load` method on the `castSession` variable's `remoteMediaClient` instance. This method call is where the media loading and playback command is actually sent to the Cast-enabled device. `remoteMediaClient` acts as the intermediary, transmitting commands from the app to the receiver. By passing the `MediaInfo` object and `MediaLoadOptions` to this method, the app specifies what to play and how it should be played, effectively initiating the streaming of video content to the Cast device.

Now, our app is ready to start casting to Google Cast devices. With that, we have finished this chapter and learned the vast possibilities of playback in Android and other connected devices.

Summary

In this chapter, we tackled the essentials of extended video playback on Android, focusing on making our app more engaging by allowing videos to play in other contexts. We covered two main areas: PiP mode and media casting, both aimed at keeping our users connected to their content, whether they're multitasking on their device or looking to enjoy videos on a larger screen.

Starting with PiP, we walked through how to enable a video to continue playing in a small window while users navigate away from the app. This section detailed everything from modifying your app's manifest to implementing PiP mode, ensuring users won't have to pause their viewing experience when they need to use another app.

Next, we shifted focus to media casting, particularly with `MediaRouter` and the Cast SDK for devices such as Google Chromecast. Here, you learned how to let users send video from their mobile device to a TV. We discussed using `MediaRouteButton` for easy device discovery and connec-

tion, as well as how to create a custom UI for users who want more control over the casting process.

By the end of this chapter, you should have understood how to implement PiP for in-app multitasking and set up casting to external devices. These skills are key to creating Android apps that offer flexible and user-friendly video playback experiences. Whether it's keeping a video running in a corner of the screen or sharing a favorite movie on a big TV, your app can now cater to various user needs, enhancing overall engagement with your video content.

And with that, we've reached the end of our journey, where we built key features for three types of apps: a messaging app, a social platform, and a video app. Each project aimed to deepen your Android and Kotlin development skills and inspire you to think about how you can apply these ideas to your own work.

Thank you for reading this book. I hope it has not only broadened your knowledge but also sparked new ideas for your projects. With the tools and techniques you've learned, you're well prepared to advance your career and start building your own innovative apps. Here's to your success in the field of mobile development – go out there and make great things!